

Towards Decentralized Feature Models

Mukelabai Mukelabai^{1,4}, Johan Martinson^{1,5}, Wardah Mahmood²
, Thorsten Berger^{1,2}, and Rafael Capilla³

¹ Ruhr University Bochum, Bochum, Germany
{mukelabai.mukelabai,johan.martinson,thorsten.berger}@rub.de

² Chalmers | University of Gothenburg, Sweden
wardahm@chalmers.se

³ Rey Juan Carlos University, Spain
rafael.capilla@urjc.es

⁴ University of Zambia, Zambia

⁵ Centiro Solutions AB, Sweden

Abstract. Feature models are foundational to variability management, yet most existing techniques assume a conceptually single feature model—even when technically partitioned across files or views. This assumption is increasingly misaligned with modular and ecosystem-scale development, where variability is distributed across subsystems owned and evolved by different teams. In this paper, we advance the notion of decentralized feature modeling, in which multiple asset-level feature models coexist, each with an explicit scope and owner, and global consistency being achieved through capability-based cross-model constraints rather than reconstruction of a monolithic model. We demonstrate how decentralized models can be structured, how capabilities express cross-team dependencies, and how distributed configuration ensures global satisfiability while preserving local autonomy. In fact, no existing modular, multi-view, or multi-level feature modeling technique fully addresses placement, independent configuration, and incremental analysis in an integrated manner. We derive requirements for decentralized modeling and identify research challenges around decomposition, capability governance, cross-model reasoning, and scalable distributed analysis.

Keywords: Feature models · Variability models · Ecosystems · Decentralization

1 Introduction

Feature models are widely used to formally represent and to analyze variability in software product lines and ecosystems, originating with Feature-Oriented Domain Analysis (FODA) [24] and consolidated in product-line engineering practice [37]. Several studies [10,11,7] demonstrate their widespread adoption and use for configuring, reasoning, and communicating about what software products can be derived from what assets. Despite decades of research and tool support [4,6,42], the predominant assumption is that variability is represented in a *single conceptual feature model*, even if technically split across multiple files or views. For instance, the Linux kernel’s Kconfig feature model distributes the declaration of

variability over more than a thousand files, but these fragments are interpreted as one global model during analysis and configuration [29,9]. Similarly, modern textual variability modeling languages, such as UVL, provide import mechanisms that ultimately yield a single feature space [42,43].

This assumption is increasingly misaligned with modern development practices. Large software systems are organized into modules, services, and packages that are owned and evolved by different teams or even organizations, ranging from traditional product lines to open, compositional platforms and ecosystems [46,9,14]. Their variability is often encoded implicitly using configuration flags, build options, environment variables, and deployment descriptors. A single global feature model is rarely maintained in practice, and when it is, it can become a bottleneck: changing one subsystem may require editing and re-validating the global feature model, and teams can hardly evolve their respective variable components or modules independently [40,8].

This motivates a paradigm shift towards more flexible feature modeling mechanisms—*decentralized feature modeling* [40,38,13,2,9]—where multiple feature models coexist, each attached to an asset or subsystem with its own scope and owner, yet collectively contributing to global configurability. Instead of reconstructing a monolithic model, consistency is achieved via explicit cross-model constraints and abstraction mechanisms such as capabilities (explained in Sec. 3).

In this short vision paper, we highlight a fundamental gap in current variability modeling: although modular, multi-view, and multi-level techniques exist, they ultimately rely on a single conceptual feature space and do not support independent placement, configuration, and evolution of multiple team-owned feature models. To address this gap, we (i) articulate the notion of decentralized feature modeling and clarify how it diverges from mere syntactic fragmentation of a global model; (ii) illustrate the approach using a mobile phone platform with capability-based cross-model dependencies and decentralized configuration; (iii) analyze existing variability mechanisms to identify which concepts translate to a decentralized setting and where conceptual and technical support is lacking; and (iv) derive requirements and research challenges for a systematic treatment of multiple interacting feature models. Our aim is not to present a full solution, but to map out the problem and lay the foundation for future research on variability in modular and ecosystem-scale systems.

2 Motivation and Definition

Traditional feature modeling techniques assume that all variability is represented in a single feature hierarchy [37,16]. Even when implemented as multiple files, as in the Linux kernel’s Kconfig system, tooling treats them as fragments of a global feature model: symbols are resolved globally, and configuration is performed in a single global space [29]. Similarly, feature modeling languages with import or include mechanisms conceptually assemble a single model for analysis and configuration [1,43].

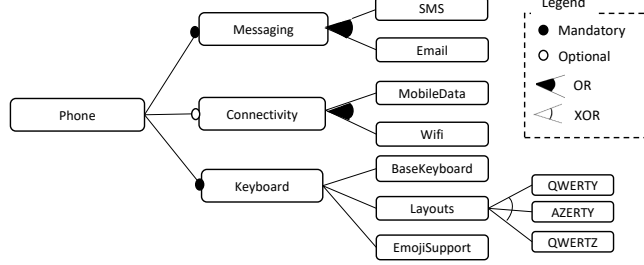


Fig. 1. A centralized phone feature model

This global perspective has two drawbacks in modular and ecosystem settings: (i) *global coupling*, since changes to the variability of one subsystem may propagate into a global model, creating coordination overhead and potential conflicts between teams, as reported in industrial case studies [8,40]; and (ii) *misalignment with code ownership*, since teams own and evolve modules, not the system as a whole. It is unnatural for them to maintain and reason about a global variability space that includes unrelated functionality [13,10].

In contrast, modern systems are often structured as microservices or packages that evolve independently. Each service or package effectively has its own ‘mini feature model,’ encoded in configuration files, environment variables, or build flags, as seen in research towards multi-level and multi-purpose modeling approaches [38,30] and multi-granularity variability management [21,44]. These implicit models are difficult to analyze and compose, yet they are central to how variability is realized in practice.

We argue that this situation calls for *decentralized feature modeling*, in which variability is explicitly captured in multiple feature models, each attached to a subsystem, rather than in a single global model.

Definition 1 (Decentralized feature modeling). *A feature modeling technique is decentralized if (i) feature models are attached to separate assets or subsystems in the codebase; (ii) each feature model has a well-defined local scope and can be evolved and configured by a responsible team; and (iii) consistency across models is ensured by explicit constraint mechanisms (e.g., capabilities, cross-model constraints), without requiring a single, permanent global feature model instance.*

This definition distinguishes decentralization from traditional modularization, where fragments are ultimately merged into a global model [3,1]. In a decentralized approach, the models remain structurally separate and interact only through explicit cross-model dependencies, similar to feature-model interfaces [41], but generalized to independently owned models.

3 Illustrative Example

We illustrate our vision using a mobile phone platform inspired by prior decomposition examples [16,17]. Traditionally, variability is captured in one centralized

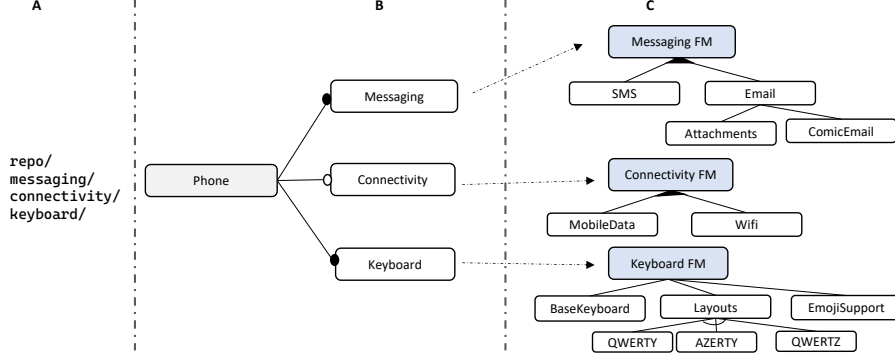


Fig. 2. Decomposing a centralized feature model into sub-feature models: (a) code base structure; (b) centralized model; (c) decentralized subsystem models.

model rooted at **Phone**, with child-features such as **Messaging**, **Connectivity**, and **Keyboard**. Figure 1 shows such a simplified centralized model.

In practice, however, implementations are modular and maintained by separate teams. Thus, rather than one global model, decentralized feature modeling attaches one feature model per subsystem, as shown in Fig. 2: Messaging, Connectivity, and Keyboard. Each model governs the variability of its *local scope*, i.e., the assets to which it is attached, similar to asset-level variability in the virtual platform [30]. Scope generalizes the notion of *feature context* [12] from individual features to entire feature models.

Capabilities as cross-model contracts. To relate decentralized models, we use *capabilities* as stable abstractions instead of direct feature references, inspired by capability matching in service composition [20]. A feature may *require* or *provide* capabilities. For example:

$$\text{Email} \Rightarrow \text{Requires}(\text{InternetConnectivity}),$$

$$\text{MobileData} \Rightarrow \text{Provides}(\text{InternetConnectivity}).$$

Similarly, Messaging may require *KeyboardInput* and *EmojiRendering*, provided by the Keyboard model. This generalizes feature-model interfaces [41] from sub-model composition to independently owned models.

Decentralised configuration. A key question in decentralized feature modeling is how independent teams know when to configure their feature models. Unlike centralized configuration, where one team selects features for the entire product, each team in a decentralized setting configures only its own model, triggered either by (i) *team-internal needs* or (ii) *system-level integration*. Internal triggers occur whenever a subsystem is released or adapted, ensuring satisfaction of local constraints, similar to multi-level configuration spaces [38] and asset-level variability in the virtual platform [30]. System-level triggers arise when a product is assembled. A global capability resolver checks whether all required capabilities are provided and informs only the teams responsible for missing provisions,

avoiding the construction of a monolithic model and aligning with work on distributed configuration and multi product lines [18,46].

We illustrate this process using the *ComicEmail* feature. Suppose a product integrator wishes to assemble a Phone supporting ComicEmail. Only the Messaging team selects features from the Messaging model, choosing:

$$\{Messaging, Email, ComicEmail\}.$$

This induces the following required capabilities:

$$req = \{KeyboardInput, InternetConnectivity, EmojiRendering\}.$$

Connectivity and Keyboard teams configure their own models independently. For example, selecting *MobileData* provides *InternetConnectivity*, while *BaseKeyboard* and *EmojiSupport* provide *KeyboardInput* and *EmojiRendering*.

A global resolver collects all provided capabilities:

$$prov = \{InternetConnectivity, KeyboardInput, EmojiRendering\},$$

and the configuration is valid iff

$$\forall c \in req : c \in prov.$$

If a team selects no feature that provides a required capability (e.g., no network feature), the resolver reports the missing capability and only that team updates its configuration. No team ever modifies another team's model.

Formally, each local feature model FM_i contributes constraints C_i , and capability requirements generate cross-model clauses such as:

$$ComicEmail \Rightarrow EmojiRendering.$$

If a capability c has providers $\{p_1, \dots, p_k\}$, global consistency requires:

$$c \Rightarrow (p_1 \vee \dots \vee p_k).$$

A decentralized configuration is valid iff

$$C = \bigcup_i C_i \cup C_{capabilities}$$

is satisfiable. Only capability clauses cross model boundaries; all other constraints remain local, enabling independent configuration with coordinated global validation.

Each decentralized feature model FM_i defines an independent local configuration space with validity condition:

$$Valid(FM_i).$$

A global decentralized configuration is valid iff all local models are individually valid and all required capabilities are satisfied:

$$Valid_{global} = \bigwedge_i Valid(FM_i) \wedge \bigwedge_{c \in Req} Provided(c).$$

Thus, decentralized feature models preserve local semantics while introducing a global capability-satisfaction condition. Unlike modular feature models, global validity does not require reconstruction of a monolithic feature hierarchy.

Cross-model interactions. Decentralized feature models do not eliminate feature interactions across models but seek to minimize direct feature-to-feature coupling. Where possible, interactions should be abstracted through capabilities rather than explicit cross-model constraints. For instance, suppose the Messaging FM contains feature *EncryptedMessaging* while a Storage FM contains *LegacyStorage*, and encrypted messaging must not be used with insecure legacy storage. Rather than expressing:

$$EncryptedMessaging \Rightarrow \neg LegacyStorage,$$

the interaction can be abstracted as:

$$EncryptedMessaging \Rightarrow Requires(SecureStorage),$$

$$LegacyStorage \Rightarrow Provides(InsecureStorage),$$

with a global incompatibility rule:

$$\neg(SecureStorage \wedge InsecureStorage).$$

This allows each model to remain locally decoupled while still enforcing global interaction constraints.

In cases where abstraction through capabilities is infeasible, explicit cross-model interaction constraints may be maintained in a separate coordination layer:

$$Valid_{global} = \bigwedge_i Valid(FM_i) \wedge C_{capabilities} \wedge C_{interactions}.$$

Here, $C_{interactions}$ captures unavoidable direct inter-model constraints, while $C_{capabilities}$ captures capability satisfaction constraints. This preserves decentralized ownership while enabling consistent reasoning over interacting feature models.

4 State of the Art and Limitations

Existing work provides several mechanisms, most of which retain a global variability perspective. *Modular feature modeling and composition* decompose models but reconstruct a single global FM for analysis [3,1]. *Multi-layer* FMs structure variability into conceptual layers [28,15], while *multi-view variability* separates stakeholder perspectives but still assumes a shared feature universe [22,5]. *Interfaces and capability-based contracts* constrain interactions between model fragments [41,25], yet do not support independently scoped and configured models. Multi-level approaches such as FORCE [38] and the virtual platform [30] distribute variability across abstraction levels and assets but rely on direct feature

Table 1. Comparison of representative variability modeling approaches.

Approach	Global FM	Indep. Models	Asset Scope	Dist. Config.
Modular FMs [3,1]	Yes	No	No	No
Multi-view FMs [22]	Yes	No	No	No
FM Interfaces [41]	Yes	Partial	No	No
FORCE [38]	Yes	Partial	Partial	No
Virtual Platform [30]	Yes	Partial	Yes	No
Decentralized FMs	No	Yes	Yes	Yes

dependencies. Table 1 summarizes the differences between representative prior approaches and decentralized feature modeling.

Scope remains largely underexplored in distributed variability modeling. The closest concept is *feature context* by Bittner et al. [12], defined at individual-feature granularity. Our notion of model scope generalizes this as the union of contexts of all features contained in a feature model.

Recent empirical evidence further motivates decentralization. Variability is rarely modular: implementations are scattered across artifacts [44], span multiple layers and granularities [21], and are fragmented across build scripts, manifests, and annotations [10,11,35,9,26,34,23,12]. Centralized FMs are therefore increasingly difficult to maintain and evolve [40,13].

Overall, three limitations emerge: (1) most approaches assume a single conceptual feature space, even when fragmented [42,4,28]; (2) model placement and scope are not first-class modeling constructs except in limited multi-level frameworks [38,30]; and (3) support for independently configured and analyzed models remains minimal [41,18].

Without explicit scoping, placement, and cross-model dependency mechanisms, systems with multiple feature models cannot be configured or evolved in a controlled manner.

5 Requirements and Research Challenges

From the example and limitations in Sec. 4, we derive key requirements for decentralized feature modeling: R1: *Explicit scope and placement*—each model must specify the assets it governs [30]. R2: *Model independence*—teams should evolve and configure models without modifying a global FM [46]. R3: *Stable cross-model abstractions*—dependencies should use capabilities rather than direct feature links [41,20]. R4: *Decentralized configuration*—local choices must combine with capability constraints to ensure global consistency [18]. R5: *Incremental analysis*—analysis should reuse results for unchanged models instead of reconstructing monolithic FMs [3,41]. R6: *Reuse and cloning*—subsystems and their models should remain reusable across products [9,14,30].

Meeting these requirements exposes several research challenges.

C1: Decomposition and scoping. Determining suitable model boundaries requires understanding architecture, cross-cutting concerns, and dependencies [39,34].

Overly coarse models reintroduce centralization, whereas overly fine models increase coordination effort. Existing techniques such as slicing and view extraction [2,45] provide partial support but do not address organizational decomposition.

C2: Structuring sub-models. Sub-models may be organized hierarchically or orthogonally. Hierarchies improve structure but risk re-centralizing variability [38], whereas orthogonal models better support independent teams but may obscure dependencies [32,33,36].

C3: Capability management. Independent teams may introduce inconsistent capability names, requiring lightweight governance such as naming conventions, catalogues, and duplicate detection [6,42].

C4: Model sizing and maintainability. Determining when a model is too large or too small currently relies on intuition. Metrics such as feature count, constraint density, and dependency degree may support decomposition and refactoring [38,11,21].

C5: Managing cross-model dependencies. Decentralization is beneficial only if cross-model constraints remain manageable. Capability interfaces help, but refactorings and heuristics for reducing unnecessary inter-model (n-to-m) links remain underdeveloped [44,26,19,31,27].

C6: Scalable distributed analysis. Decentralized modeling requires modular SAT encodings and incremental reasoning over local FMs plus capability constraints [38,40]. Rather than solving one monolithic SAT problem, analysis may proceed as:

$$SAT(FM_1), SAT(FM_2), \dots, SAT(FM_n), SAT(C_{capabilities}).$$

This may reduce recomputation costs during evolution, although empirical validation remains future work.

Overall, decentralized feature modeling demands new decomposition, governance, and analysis techniques that treat multiple interacting feature models as first-class entities.

6 Conclusion

Decentralized feature models extend conventional feature models with independent local semantics, decentralized ownership, and capability-based cross-model composition. Global configurations emerge from combining local configurations under capability constraints rather than reconstructing a monolithic model.

In this vision paper, we introduced decentralized feature models, illustrated them through a running example, and showed how capabilities support distributed configuration. Our review indicates that existing approaches provide partial support but do not jointly address scope, placement, independent configuration, and incremental analysis.

We further outlined key requirements and research challenges toward scalable, modular variability engineering, and hope this work encourages further exploration of decentralized variability modeling as a first-class paradigm.

References

1. Acher, M., Collet, P., Lahire, P., France, R.B.: Composing feature models. In: van den Brand, M., Gasevic, D., Gray, J. (eds.) *Software Language Engineering, Second International Conference, SLE 2009, Denver, CO, USA, October 5-6, 2009, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 5969, pp. 62–81. Springer (2009)
2. Acher, M., Collet, P., Lahire, P., France, R.B.: Slicing feature models. In: *Proceedings of the 26th IEEE/ACM International Conference on Automated Software Engineering* (2011)
3. Bagheri, E., Ensan, F., Gasevic, D., Boskovic, M.: Modular feature models: Representation and configuration. *Journal of Research and Practice in Information Technology* **43**(2), 109–140 (2011)
4. Bąk, K., Diskin, Z., Antkiewicz, M., Czarnecki, K., Wąsowski, A.: Clafer: unifying class and feature modeling. *Software & Systems Modeling* **15**, 811–845 (2016)
5. de beeck, S.O., van Dooren, M., Lagaisse, B., Joosen, W.: Modularity and variability of distributed software architectures through multi-view refinement of ao-connectors. *Transactions on Aspect-Oriented Software Development* **10**, 109–147 (2013)
6. Berger, T., Collet, P.: Usage scenarios for a common feature modeling language. In: *Proceedings of the 23rd International Systems and Software Product Line Conference, SPLC 2019, Volume B, Paris, France, September 9-13, 2019. ACM* (2019)
7. Berger, T., Lettner, D., Rubin, J., Grünbacher, P., Silva, A., Becker, M., Chechik, M., Czarnecki, K.: What is a feature? a qualitative study of features in industrial software product lines. In: *SPLC* (2015)
8. Berger, T., Nair, D., Rublack, R., Atlee, J.M., Czarnecki, K., Wasowski, A.: Three cases of feature-based variability modeling in industry. In: *Proceedings of the 17th International Conference of Model-Driven Engineering Languages and Systems (MODELS)*. pp. 302–319 (2014)
9. Berger, T., Pfeiffer, R.H., Tartler, R., Dienst, S., Czarnecki, K., Wąsowski, A., She, S.: Variability mechanisms in software ecosystems. *Information and Software Technology* **56**(11), 1520–1535 (2014)
10. Berger, T., Rublack, R., Nair, D., Atlee, J.M., Becker, M., Czarnecki, K., Wąsowski, A.: A survey of variability modeling in industrial practice. In: *Proceedings of the 7th international workshop on variability modelling of software-intensive systems* (2013)
11. Berger, T., She, S., Lotufo, R., Wasowski, A., Czarnecki, K.: A study of variability models and languages in the systems software domain. *IEEE Transactions on Software Engineering* **39**(12), 1611–1640 (2013)
12. Bittner, P.M., Schultheiß, A., Thüm, T., Kehrer, T., Young, J.M., Linsbauer, L.: Feature trace recording. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2021)
13. Boutkova, E.: Experience with variability management in requirement specifications. In: de Almeida, E.S., Kishi, T., Schwanninger, C., John, I., Schmid, K. (eds.) *Software Product Lines - 15th International Conference, SPLC 2011, Munich, Germany, August 22-26, 2011*. pp. 303–312. IEEE Computer Society (2011)
14. Businge, J., Openja, M., Nadi, S., Berger, T.: Reuse and maintenance practices among divergent forks in three software ecosystems. *Empirical Software Engineering* **27**(2), 54 (2022)

15. Cañete, A., Amor, M., Fuentes, L.: Supporting iot applications deployment on edge-based infrastructures using multi-layer feature models. *J. Syst. Softw.* **183**, 111086 (2022)
16. Capilla, R., Bosch, J., Kang, K.C. (eds.): *Systems and Software Variability Management - Concepts, Tools and Experiences*. Springer (2013)
17. Capilla, R., Ortiz, Ó., Hinchey, M.: Context variability for context-aware systems. *Computer* **47**(2), 85–87 (2014)
18. Dhungana, D., Seichter, D., Botterweck, G., Rabiser, R., Grunbacher, P., Benavides, D., Galindo, J.A.: Configuration of multi product lines by bridging heterogeneous variability modeling approaches. In: *Proceedings of the 15th International Software Product Line Conference* (2011)
19. Fischer, S., Linsbauer, L., Lopez-Herrejon, R.E., Egyed, A.: The ecco tool: Extraction and composition for clone-and-own. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. vol. 2, pp. 665–668. IEEE (2015)
20. Gao, X., Yang, J., Papazoglou, M.P.: The capability matching of web services. In: *Fourth International Symposium on Multimedia Software Engineering, 2002. Proceedings*. pp. 56–63. IEEE (2002)
21. Horcas, J.M., Cortiñas, A., Fuentes, L., Luaces, M.R.: Combining multiple granularity variability in a software product line approach for web engineering. *Inf. Softw. Technol.* **148**, 106910 (2022)
22. Hubaux, A., Heymans, P., Schobbens, P.Y., Deridder, D., Abbasi, E.K.: Supporting multiple perspectives in feature-based configuration. *Software & Systems Modeling* **12**, 641–663 (2013)
23. Ji, W., Berger, T., Antkiewicz, M., Czarnecki, K.: Maintaining feature traceability with embedded annotations. In: *Proceedings of the 19th International Conference on Software Product Line*. pp. 61–70 (2015)
24. Kang, K.C., Cohen, S.G., Hess, J.A., Novak, W.E., Peterson, A.S.: *Feature-oriented domain analysis (foda) feasibility study*. Tech. rep., Carnegie-Mellon Univ Pittsburgh Pa Software Engineering Inst (1990)
25. Kästner, C., Apel, S., Ostermann, K.: The road to feature modularity? In: Schaefer, I., John, I., Schmid, K. (eds.) *Proceedings of the 15th Software Product Lines International Conference, SPLC 2011, Munich, Germany, August 22-26, 2011. Workshop Proceedings (Volume 2)*. p. 5. ACM (2011)
26. Krüger, J., Pinnecke, M., Kenner, A., Kruczek, C., Benduhn, F., Leich, T., Saake, G.: Composing annotations without regret? practical experiences using featurec. *Softw. Pract. Exp.* **48**(3), 402–427 (2018)
27. Lapeña, R., Ballarin, M., Cetina, C.: Towards clone-and-own support: locating relevant methods in legacy products. In: *Proceedings of the 20th International Systems and Software Product Line Conference*. pp. 194–203 (2016)
28. Lettner, M., Rodas-Silva, J., Galindo, J.A., Benavides, D.: Automated analysis of two-layered feature models with feature attributes. *J. Comput. Lang.* **51**, 154–172 (2019)
29. Lotufo, R., She, S., Berger, T., Czarnecki, K., Wasowski, A.: Evolution of the linux kernel variability model. In: Bosch, J., Lee, J. (eds.) *Software Product Lines: Going Beyond - 14th International Conference, SPLC 2010, Jeju Island, South Korea, September 13-17, 2010. Proceedings. Lecture Notes in Computer Science*, vol. 6287, pp. 136–150. Springer (2010)
30. Mahmood, W., Strüber, D., Berger, T., Lämmel, R., Mukelabai, M.: Seamless variability management with the virtual platform. In: *Proceedings of the 43rd International Conference on Software Engineering* (2021)

31. Martinez, J., Ziadi, T., Bissyandé, T.F., Klein, J., Le Traon, Y.: Bottom-up technologies for reuse: automated extractive adoption of software product lines. In: 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C). pp. 67–70. IEEE (2017)
32. Mens, K., Capilla, R., Cardozo, N., Dumas, B.: A taxonomy of context-aware software variability approaches. In: Fuentes, L., Batory, D.S., Czarnecki, K. (eds.) Companion Proceedings of the 15th International Conference on Modularity, Málaga, Spain, March 14 - 18, 2016. pp. 119–124. ACM (2016)
33. Mens, K., Capilla, R., Hartmann, H., Kropf, T.: Modeling and managing context-aware systems’ variability. *IEEE Softw.* **34**(6), 58–63 (2017)
34. Mezini, M., Ostermann, K.: Variability management with feature-oriented programming and aspects. In: SIGSOFT Software Engineering Notes, 29(6). pp. 127–136 (2004)
35. Nadi, S., Berger, T., Kästner, C., Czarnecki, K.: Where do configuration constraints stem from? an extraction approach and an empirical study. *IEEE Transactions on Software Engineering* **41**(8), 820–841 (2015)
36. Nesic, D., Krueger, J., Stanculescu, S., Berger, T.: Principles of feature modeling. In: 27th ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE) (2019)
37. Pohl, K., Böckle, G., van der Linden, F.: *Software Product Line Engineering - Foundations, Principles, and Techniques*. Springer (2005)
38. Rabiser, D., Prähofer, H., Grünbacher, P., Petruzelka, M., Eder, K., Angerer, F., Kromoser, M., Grimmer, A.: Multi-purpose, multi-level feature modeling of large-scale industrial software systems. *Software & Systems Modeling* **17**, 913–938 (2018)
39. Robillard, M.P., Murphy, G.C.: Representing concerns in source code. *ACM Transactions on Software Engineering and Methodology (TOSEM)* **16**(1), 3–es (2007)
40. Schmid, K.: Variability modeling for distributed development—a comparison with established practice. In: *Software Product Lines: Going Beyond: 14th International Conference, SPLC 2010, Jeju Island, South Korea, September 13–17, 2010. Proceedings 14*. pp. 151–165. Springer (2010)
41. Schröter, R., Krieter, S., Thüm, T., Benduhn, F., Saake, G.: Feature-model interfaces: the highway to compositional analyses of highly-configurable systems. In: *Proceedings of the 38th International Conference on Software Engineering*. pp. 667–678 (2016)
42. Sundermann, C., Feichtinger, K., Engelhardt, D., Rabiser, R., Thüm, T.: Yet another textual variability language? a community effort towards a unified language. In: *Proceedings of the 25th ACM International Systems and Software Product Line Conference-Volume A* (2021)
43. Sundermann, C., Heß, T., Engelhardt, D., Arens, R., Herschel, J., Jedelhauser, K., Jutz, B., Krieter, S., Schaefer, I.: Integration of uvl in featureide. In: *Proceedings of the 25th ACM International Systems and Software Product Line Conference-Volume B*. pp. 73–79 (2021)
44. Tärnava, X., Collet, P.: A framework for managing the imperfect modularity of variability implementations. *Journal of Computer Languages* **61**, 100998 (2020)
45. Urli, S., Bergel, A., Blay-Fornarino, M., Collet, P., Mosser, S.: A visual support for decomposing complex feature models. In: *Proceedings of the 3rd Working Conference on Software Visualization* (2015)
46. Van Gorp, J., Prehofer, C.: From spls to open, compositional platforms. In: *Dagstuhl Seminar Proceedings. Schloss Dagstuhl-Leibniz-Zentrum für Informatik* (2008)